Code Explanation

Unit Tests for Data Extraction Module

The code provided is a set of unit tests for a data extraction module written in Python using the Pytest framework. It tests the functionality of three main functions: `get_spark_session`, `read_source_table`, and `extract_source_data`. These tests ensure that the data extraction module is working as expected.

Overview of the Code

The code is a collection of unit tests that validate the functionality of the data extraction module. It uses Pytest fixtures and mocking to isolate dependencies and test the functions in isolation. The tests cover the creation of a SparkSession, reading data from a source table, and extracting data from multiple source tables.

Step 1: Creating a Mock SparkSession
The test suite starts by defining a Pytest fixture `mock_spark` that creates a mock SparkSession object. This fixture is used throughout the tests to mock the SparkSession dependency.

```
@pytest.fixture
def mock_spark():
    spark = MagicMock()
    reader = MagicMock()
    spark.read = reader
    reader.format.return_value = reader
    reader.option.return_value = reader
    reader.load.return_value = MagicMock()
    return spark
```

Step 2: Testing get_spark_session Function
The `test_get_spark_session` function tests that the `get_spark_session` function returns a valid SparkSession object. It uses the `patch.object` function from the `unittest.mock` module to mock the `SparkSession.builder` object.

```
def test_get_spark_session():
    with patch.object(SparkSession, 'builder') as mock_builder:
        mock_session = MagicMock()
        mock_builder.appName.return_value = mock_builder
        mock_builder.config.return_value = mock_builder
        mock_builder.getOrCreate.return_value = mock_session

        session = get_spark_session()

        assert session == mock_session
        mock_builder.appName.assert_called_once_with("E2E_BC_K2H_DAT
A_EXTRACTION")
        assert mock_builder.config.call_count >= 2
        mock_builder.getOrCreate.assert_called_once()
```

Step 3: Testing read_source_table Function
The `test_read_source_table` function tests that the `read_source_table` function correctly reads data

from a source table using the provided SparkSession object. It uses the `patch` function to mock the `DB_CONFIG` and `SOURCE_TABLES` dictionaries.

```
def test_read_source_table(mock_spark):
    with patch('src.extraction.DB_CONFIG', {
        "source": {
            "jdbc_url": "jdbc:test",
            "user": "user",
            "password": "password",
            "driver": "test.driver"
        }
    }), patch('src.extraction.SOURCE_TABLES', {
        "TEST_TABLE": "schema.TEST_TABLE"
    }):
        df = read_source_table(mock_spark, "TEST_TABLE")

        mock_spark.read.format.assert_called_once_with("jdbc")
        mock_spark.read.format().option.assert_any_call("url", "jdbc
:test")
        mock_spark.read.format().option.assert_any_call("dbtable", "
schema.TEST_TABLE")
        mock_spark.read.format().option.assert_any_call("user", "use
r")
        mock_spark.read.format().option.assert_any_call("password",
"password")
        mock_spark.read.format().option.assert_any_call("driver", "t
est.driver")
        mock_spark.read.format().option().option().option().option()
.option().load.assert_called_once()
```

Step 4: Testing extract_source_data Function
The `test_extract_source_data` function tests that the `extract_source_data` function correctly extracts data from multiple source tables using the provided SparkSession object. It uses the `patch` function to mock the `SOURCE_TABLES` dictionary and the `read_source_table` function.

```
def test_extract_source_data(mock_spark):
    with patch('src.extraction.SOURCE_TABLES', {
        "TABLE1": "schema.TABLE1",
        "TABLE2": "schema.TABLE2"
    }), patch('src.extraction.read_source_table') as mock_read:
        mock_df1 = MagicMock()
        mock_df2 = MagicMock()
        mock_read.side_effect = [mock_df1, mock_df2]

        result = extract_source_data(mock_spark)

        assert mock_read.call_count == 2
        assert result == {"TABLE1": mock_df1, "TABLE2": mock_df2}
```